MLOps Assignment 2 - Final Report

Name	RIYAZ AHAMED. M
Student ID	2024AC05094
Subject	MLOps (S1-25_AIMLCZG523)
Assignment	Assignment 2
Title	Cats vs Dogs Image Classification MLOps

Introduction

This project implements an end-to-end MLOps pipeline for Cats vs Dogs binary image classification.

The pipeline covers data preprocessing and versioning, CNN model training, MLflow experiment tracking, FastAPI inference, testing, Docker containerization, GitHub Actions CI/CD, GHCR image publishing, Docker Compose, smoke testing, and basic monitoring.

Tools Used

Component	Technology
Source code	Git / GitHub
Dataset versioning	DVC
Model	PyTorch
Experiment tracking	MLflow
API	FastAPI / Uvicorn
Testing	Pytest
Containerization	Docker
CI/CD	GitHub Actions
Container registry	GHCR
Deployment	Docker Compose

Model Development & Experiment Tracking

Dataset and Preprocessing

The project uses a Cats vs Dogs image dataset containing **24,998 images**:

- Cats: 12,499
- Dogs: 12,499

Images are converted to RGB and resized to **224 × 224**.

An 80/10/10 deterministic split is used with random seed `42`.

Split	Total	Cats	Dogs
Raw	24,998	12,499	12,499
Train	19,998	9,999	9,999
Validation	2,498	1,249	1,249
Test	2,502	1,251	1,251

One augmented copy of each training image is also generated.

DVC is used to track the dataset and preprocessing pipeline.

Baseline CNN

A PyTorch `BaselineCNN` was implemented with three convolutional blocks followed by fully connected layers and dropout.

The model uses:

- `BCEWithLogitsLoss`
- Adam optimizer
- Learning rate: `0.001`
- Batch size: `32`
- Input size: `224 × 224 RGB`

The final model was trained for **10 epochs** on CPU.

Training Results

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1	0.6082	66.54%	0.5460	72.22%
2	0.5015	75.88%	0.4743	78.38%
3	0.4216	80.92%	0.4235	79.94%
4	0.3501	84.84%	0.4300	81.18%
5	0.2704	88.51%	0.4585	79.18%
6	0.2032	91.85%	0.4736	81.75%
7	0.1468	94.41%	0.5911	82.07%
8	0.1082	95.86%	0.5756	82.43%
9	0.0861	96.79%	0.7284	82.19%
10	0.0670	97.57%	0.8804	82.19%

The trained model is saved as:

```
artifacts/baseline_cnn.pt
```

Training metrics are stored in:

```
artifacts/training_metrics.json
```

Training Plots

Training and validation accuracy, as well as training and validation loss, are plotted using the metrics stored in `training_metrics.json`.

The plots are generated using:

```
        python scripts/plot_results.py
```

The generated plots are saved in:

```
artifacts/plots/
├── training_validation_accuracy.png
└── training_validation_loss.png
```

MLflow Tracking

MLflow tracks the CNN training experiment under:

```
cats-vs-dogs-baseline
```

A local SQLite database is used as the MLflow tracking backend. Training parameters and training/validation metrics are recorded for the experiment.

MLflow can be started locally using:

```
        mlflow ui --backend-store-uri sqlite:///mlflow.db
```

The MLflow UI can then be opened in the browser to view the experiment, runs, parameters, metrics, and artifacts.

Model Packaging & Containerization

FastAPI Inference

A FastAPI service was implemented to serve the trained CNN model.

Available endpoints:

Method	Endpoint	Purpose
GET	`/`	API information
GET	`/health`	API/model health

POST	`/predict`	Cat/Dog prediction
GET	`/metrics`	Request and latency metrics

The uploaded image is converted to RGB, resized to 224 × 224, converted to a tensor, and normalized before inference.

The API can be started locally using:

```
uvicorn api.main:app --reload
```

This starts the FastAPI application with automatic reload enabled during development.

The interactive Swagger API documentation is available at:

```
http://127.0.0.1:8000/docs
```

The `/docs` page can be used to view the available endpoints and test API requests directly from the browser.

The `/health` endpoint is used to check whether the API and model are available:

```
http://127.0.0.1:8000/health
```

The `/metrics` endpoint provides basic monitoring information such as total requests, requests by endpoint, and average, minimum, and maximum latency:

```
http://127.0.0.1:8000/metrics
```

Each API response also includes processing time through the `X-Process-Time-ms` response header.

Docker

The FastAPI service is containerized using a Python 3.11 slim base image.

The Docker image includes the application, required dependencies, and trained model artifact, and exposes port `8000`.

The image is published to GitHub Container Registry as:

```
ghcr.io/2024ac05094-riyaz-bits/mlops-assignment-2:latest
```

Docker installation is restricted on the development office laptop, so the Docker image could not be built or run locally on that machine. The Docker configuration and published GHCR image are included as part of the project.

CI/CD

Automated Testing

Pytest tests cover preprocessing and model inference, including image format/size, model

loading, and model output.

The test suite was executed successfully.

GitHub Actions

GitHub Actions is used to automate testing, Docker image creation, and image publishing.

The Docker image is published to GitHub Container Registry (GHCR).

Docker Compose and CD

Docker Compose uses the published GHCR image and exposes the FastAPI service on port `8000`.

The CD workflow pulls the published image, starts the application using Docker Compose, checks the API health, performs smoke testing, and verifies the container status.

Health endpoint:

```
GET /health
```

Root endpoint:

```
GET /
```

Monitoring and Model Performance

Request Logging and Counters

The FastAPI service records request and response information and maintains basic request counters by endpoint.

Example:

```
{
  "total_requests": 1,
  "requests_by_endpoint": {
    "GET /": 1
  }
}
```

Latency Tracking

The `/metrics` endpoint provides total request count and latency statistics including average, minimum, maximum, and per-endpoint latency.

Each API response also includes processing time through:

```
X-Process-Time-ms
```

Model Performance

The trained model was evaluated on the **2,502-image test set**.

Results:


```
{
  "test_samples": 2502,
  "accuracy": 0.8241,
  "precision": 0.8348,
  "recall": 0.8082,
  "f1_score": 0.8213
}
```


Metric	Score
Test Samples	2,502
Accuracy	82.41%
Precision	83.48%
Recall	80.82%
F1-score	82.13%

The results are stored in:

```
artifacts/model_performance.json
```

A model performance plot is also generated from the same JSON file:

```
artifacts/plots/model_performance.png
```

Project Structure

```
MLOps-Assignment-2/
│
├── data/
│   ├── raw/
│   └── processed/
│
├── src/
│   ├── data/
│   └── models/
│
├── api/
│   └── main.py
│
├── tests/
│   ├── test_preprocessing.py
│   └── test_inference.py
```

```
│
├── scripts/
│   ├── preprocess.py
│   ├── train.py
│   ├── evaluate.py
│   └── plot_results.py
│
├── artifacts/
│   ├── baseline_cnn.pt
│   ├── model_performance.json
│   ├── training_metrics.json
│   └── plots/
│       ├── training_validation_accuracy.png
│       ├── training_validation_loss.png
│       └── model_performance.png
│
├── .github/
│   └── workflows/
│
├── Dockerfile
├── .dockerignore
├── docker-compose.yml
├── requirements.txt
├── dvc.yaml
├── dvc.lock
├── .gitignore
└── README.md
```

Conclusion

The project demonstrates an end-to-end MLOps workflow from dataset preparation and versioning through model training, experiment tracking, API serving, testing, containerization, CI/CD, and monitoring.

The final 10-epoch CNN achieved **82.41% accuracy**, **83.48% precision**, **80.82% recall**, and **82.13% F1-score** on the test dataset.

The training and model performance results are also visualized using plots generated directly from the stored JSON metrics.

Repository

```
https://github.com/2024ac05094-Riyaz-Bits/MLOps-Assignment-2
```

Docker Image

```
ghcr.io/2024ac05094-riyaz-bits/mlops-assignment-2:latest
```